06 22313 Computational Tools for Modelling and Analysis

Identifying Groups in Data

Dr Iain Styles (I.B.Styles@bham.ac.uk)

School of Computer Science, University of Birmingham

November 2 2015

Introduction

Measures of Similarity

Normalisation

Linkage

The k-means Algorithm

Agglomerative Clustering

Other Clustering Methods

References

Outline

Introduction

Measures of Similarity

Normalisation

Linkage

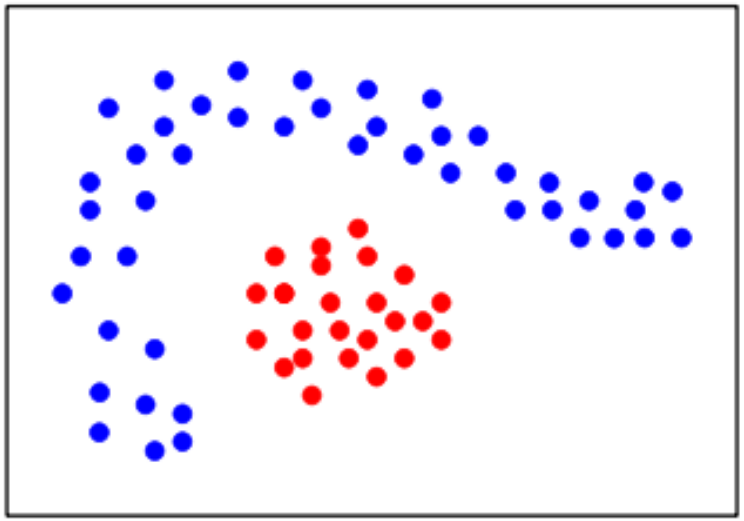
The k-means Algorithm

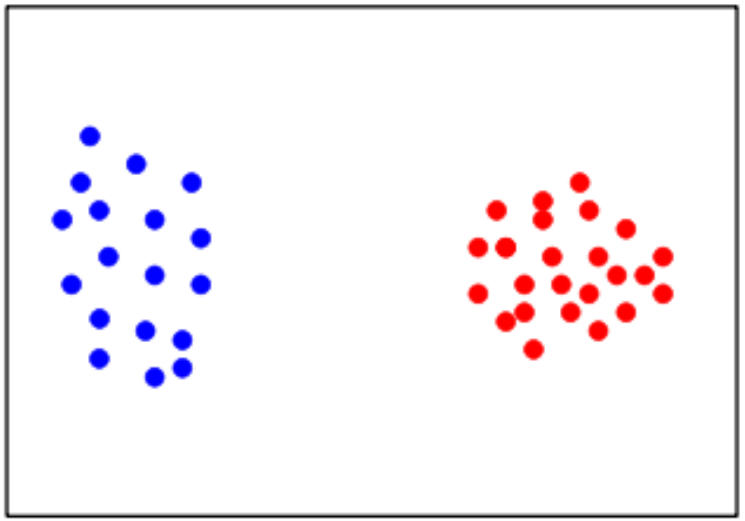
Agglomerative Clustering

Other Clustering Methods

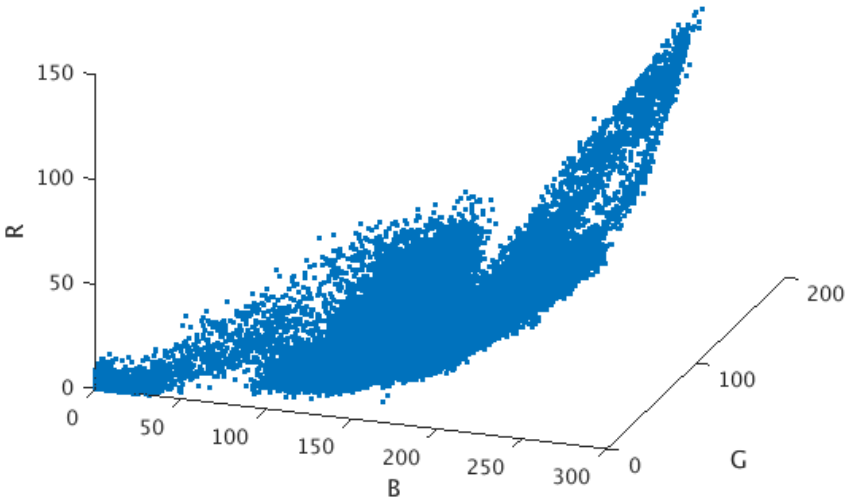
Introduction

- Fitting lines to data works well if a line is a good description of your data!
- There are many cases where this is not the case, for example when the data is composed of groups of points.
- We would want to identify what these groups were rather than just trying to fit some line to them.





Why might data be clustered?



General Approaches to Clustering

- Hundreds of clustering algorithms,
- Many assume something about the data, eg clusters are normally distributed.
- Most are based on grouping points that are "similar"
- Outcome is sets (groups) of points that by some measure "belong together".
- Main approaches are:
  - Bottom-up
  - Top-down

Measures of Similarity

- The key issue in clustering is how to measure "similarity"
- Two important issues:
  - What is the appropriate measure of similarity of two data points?
  - How do you measure the similarity of groups of points?

Introduction

Measures of Similarity

Normalisation

Linkage

The k-means Algorithm

Agglomerative Clustering

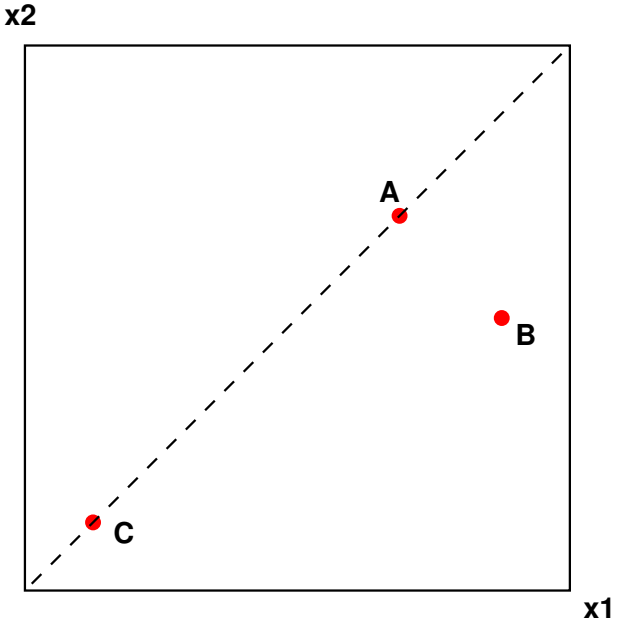
Other Clustering Methods

References

Euclidean Distance

- Intuition: similarity $\leftrightarrow$ "closeness"
- So measure the distance between points
- Most common is **Euclidean distance**.
  - Given data points $\mathbf{x}^{(1)}$ and $\mathbf{x}^{(2)}$:

$$d_{\text{euc}}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) = \sqrt{\sum_{i=1}^{M} \left[x_i^{(1)} - x_i^{(2)}\right]^2}$$



Which points are closest/most similar?

Euclidean Distance

- A and B are closest in the Euclidean sense
- But A and C are also similar in a different sense: direction
- A and C are in the "direction" from the original but are scaled
  - In a colour space, A and C are the same hue but A is brighter.
  - B is similar brightness as A, but a different hue.
- This type of similarity is captured by a different similarity (distance) measure

Cosine Distance

$$d_{\cos}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) = \cos^{-1}\left(\frac{\mathbf{x}^{(1)} \cdot \mathbf{x}^{(2)}}{\left|\left|\mathbf{x}^{(1)}\right|\right|_2 \left|\left|\mathbf{x}^{(2)}\right|\right|_2}\right)$$

- Cosine distance between A and C is zero
  - Both at the same angle relative to the origin
  - Captured by this metric
- Choice of similarity measure is highly dependent on the nature of your dataset and on what properties you want to capture in your clusters
- Not the only choices: also correlation, Manhattan and Mahalanobis distances.

Normalisation

- The same considerations also lead us to consider "normalising" each item of data
- For example, we could normalise the points all have the same length:

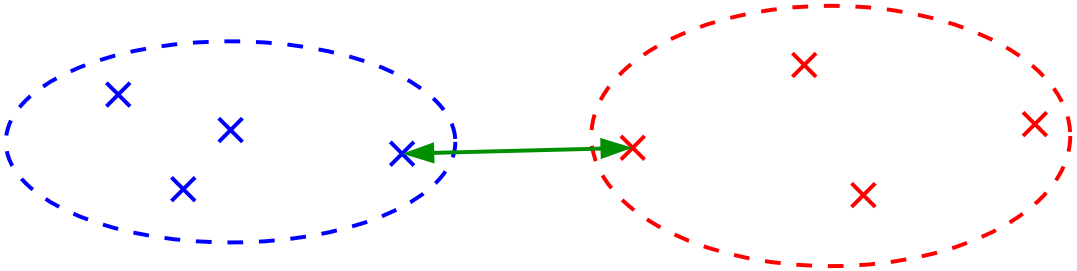
$$\sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\mathbf{x}^{\mathrm{T}}\mathbf{x}} = 1 \quad \mapsto \quad \mathbf{x} \leftarrow \frac{\mathbf{x}}{\sqrt{\mathbf{x}^{\mathrm{T}}\mathbf{x}}}$$

.

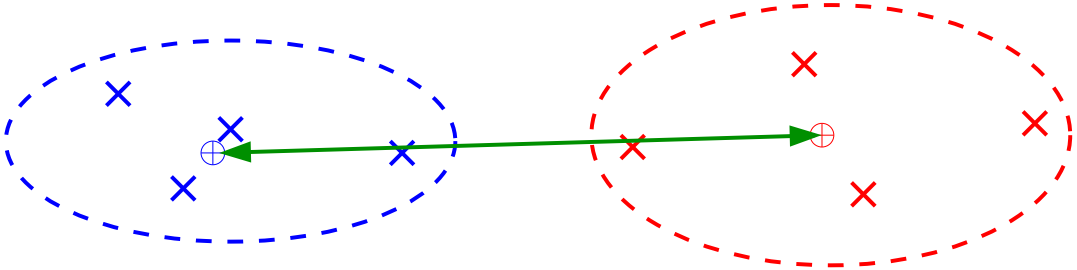
- Removes global "intensity" differences from the data.
- Draws all points radially onto the unit circle.
- A and C now co-incident
- Whether you should do this depends on what properties you wish to cluster against.

Linkage

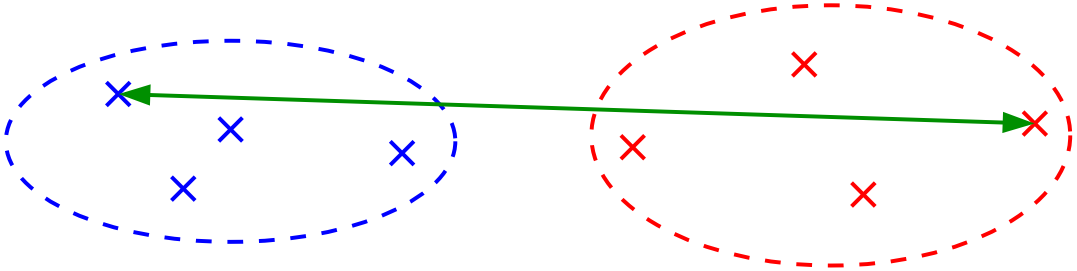
- How do we measure similarities between groups?
  - Single Linkage



  - Average Linkage

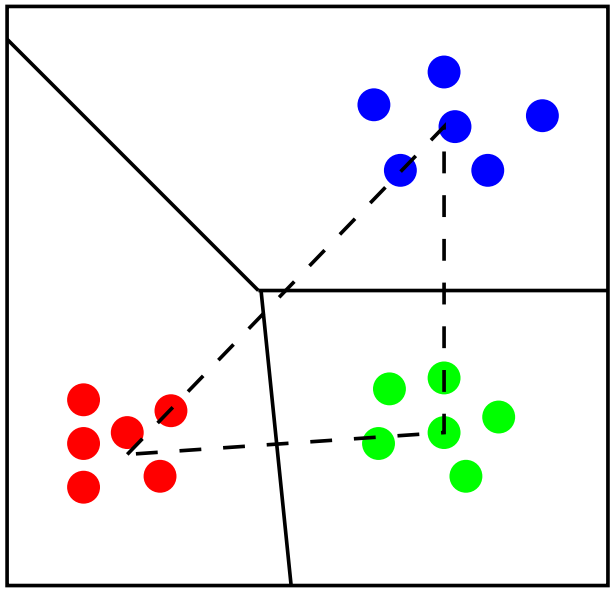


  - Complete Linkage



The k-means Algorithm

- Perhape the most commonly used clustering technique
- Partitions N data points into $k < N$ clusters
- Each data point belongs to the "nearest" cluster (nearest mean)
- Cluster boundaries are straight lines bisect those connecting the means
- Regions define by these lines define the clusters
- We will use Euclidean distance here, but other distance measures can be used.



The k-means Algorithm

- Number of clusters k is specified by the "user".
  1) First, randommly assign every data point to one of the clusters.
  2) Compute the mean position (in the data space) of each cluster
  3) For each data point, calculate distance to mean of each cluster
  4) Re-assign points to their nearest cluster.
  5) Return to step 2 and repeat until the clusters are not changing.

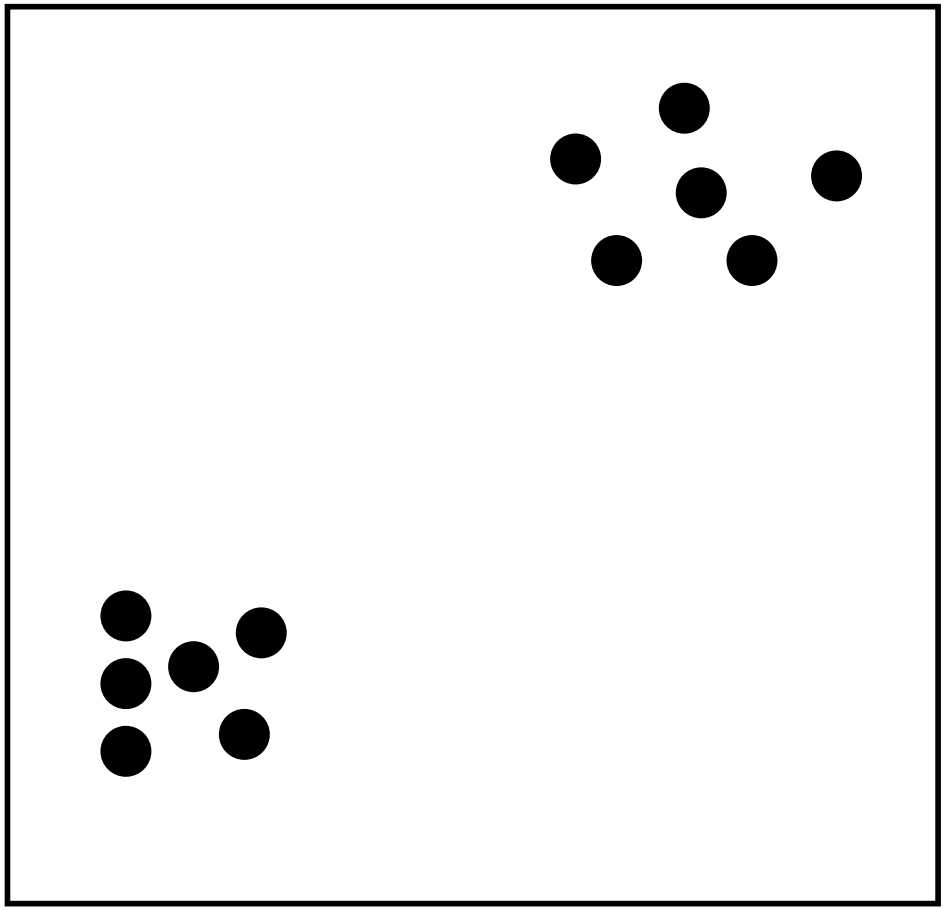
Notes about k-means

There are a few points to note:

- Initial random seeding $\rightarrow$ repeats needed
- Have to "guess" the number of clusters.
  - Too many, and you will split what should be a single cluster.
  - Too few, and you will combine clusters that should be separate, or splitting clusters and merge with others.
- This is a computationally "hard" algorithm (in the jargon, it is NP-hard).
  - The number of distance calculations increases rapidly with the number of clusters and the number of data points so it can be very costly for large datasets.
- Works with all distance measures, although the "shape" of the clusters will be less intuitive.
- Linkage strategies are not required as the only distances computed are between the points and the means.

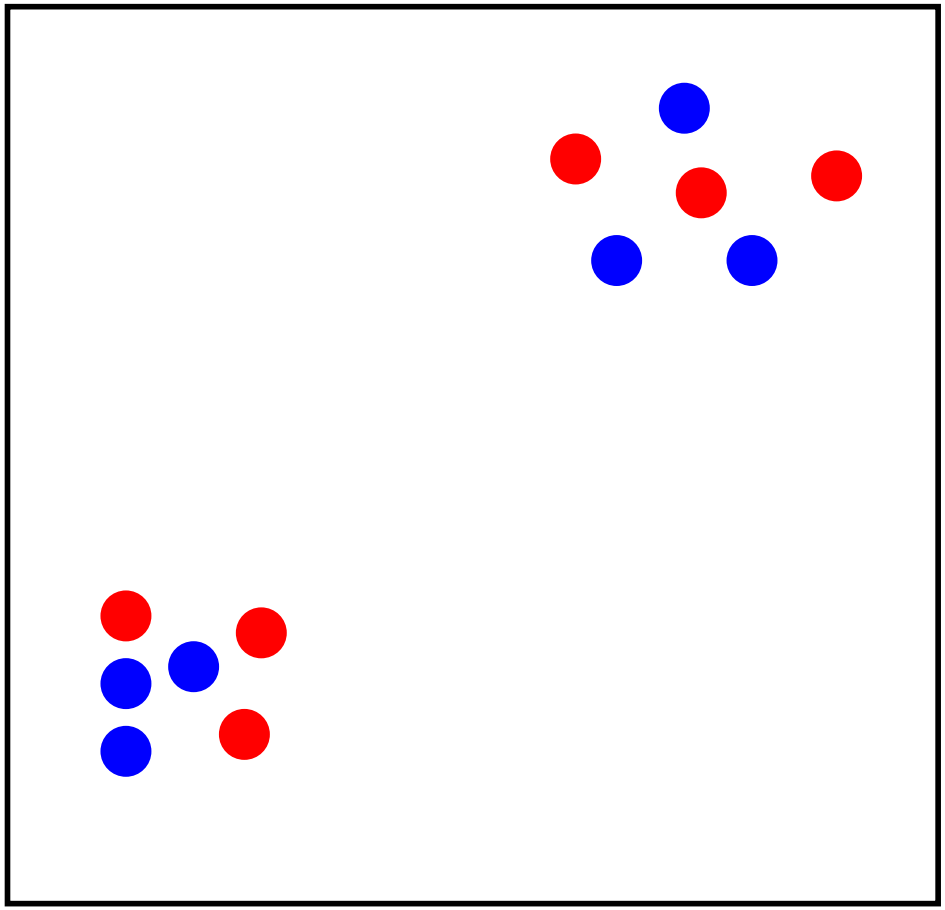
The k-means Algorithm

Initial data points



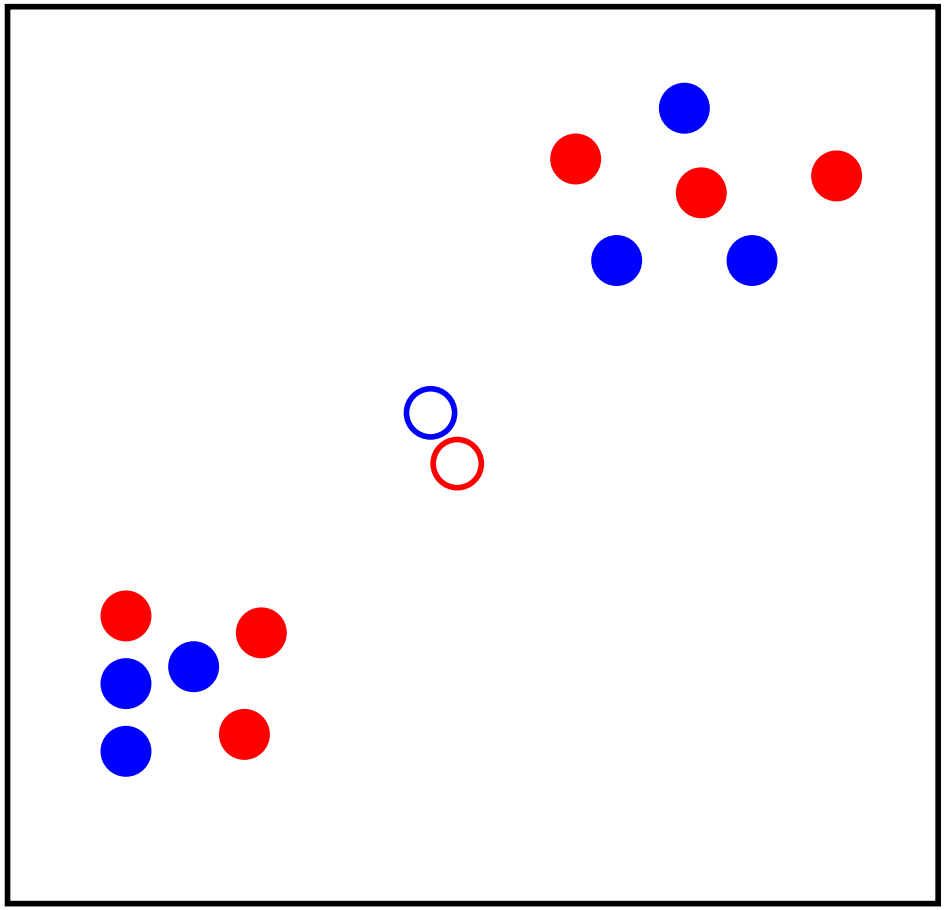
The k-means Algorithm

Randomly assign points to groups



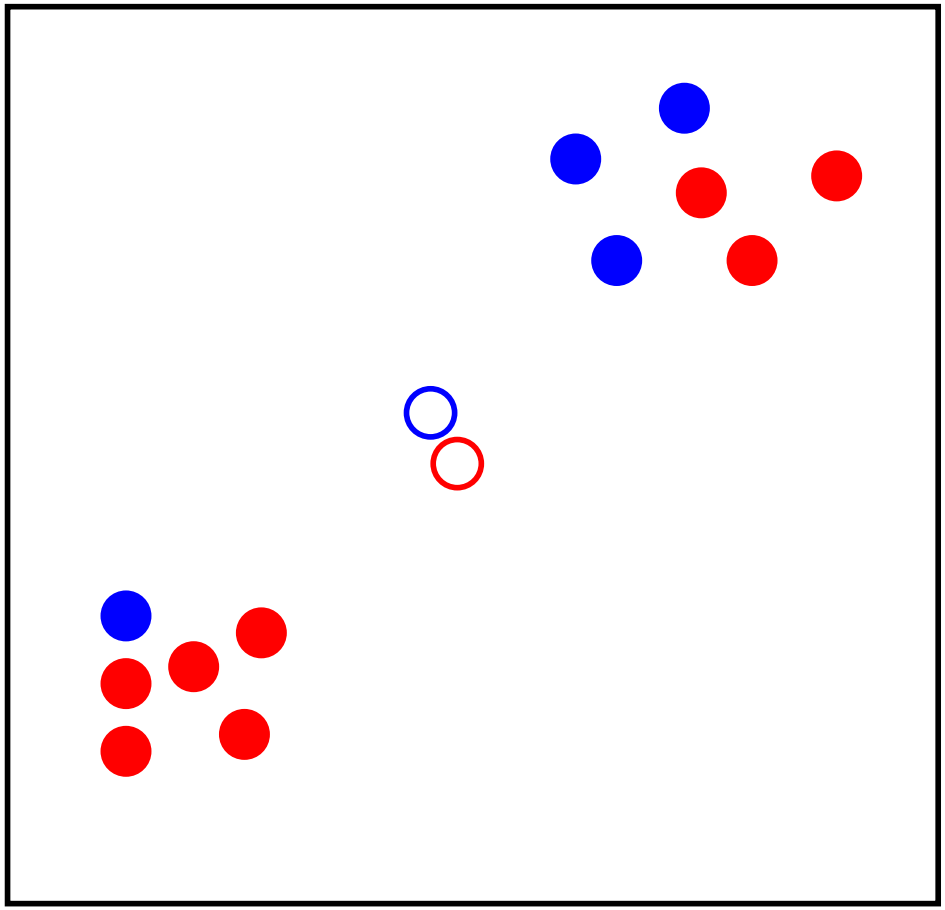
The k-means Algorithm

Compute group average



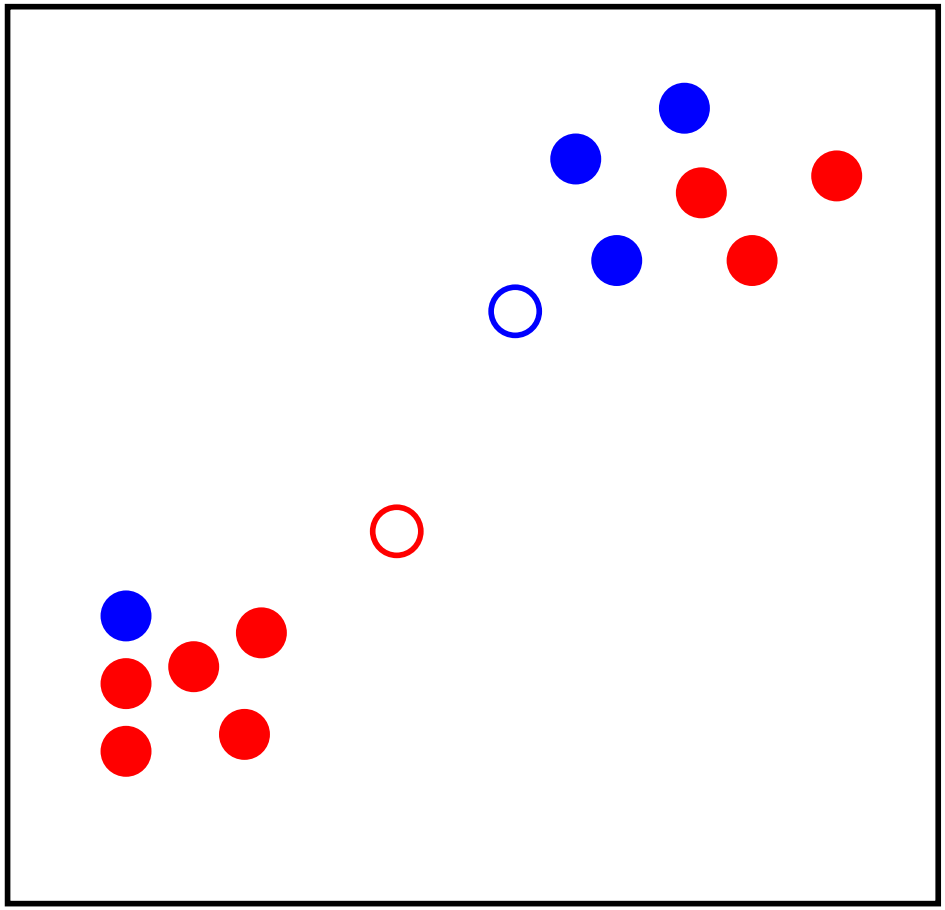
The k-means Algorithm

Re-assign points to groups



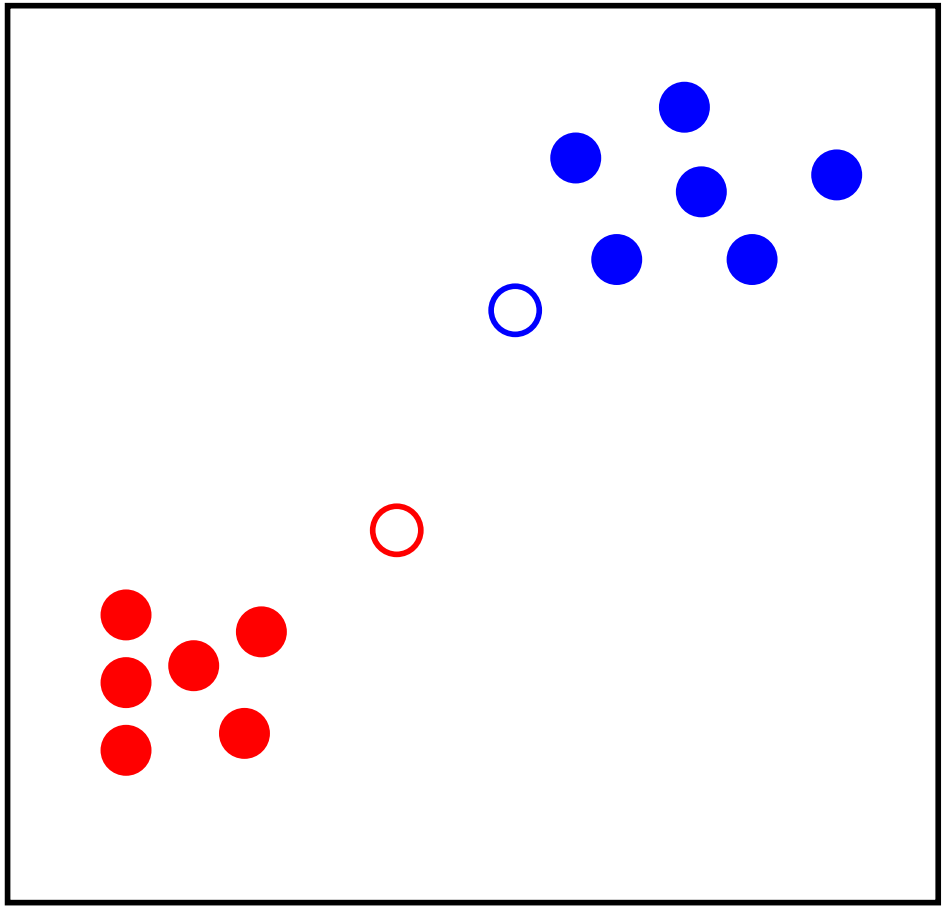
The k-means Algorithm

Re-compute group averages



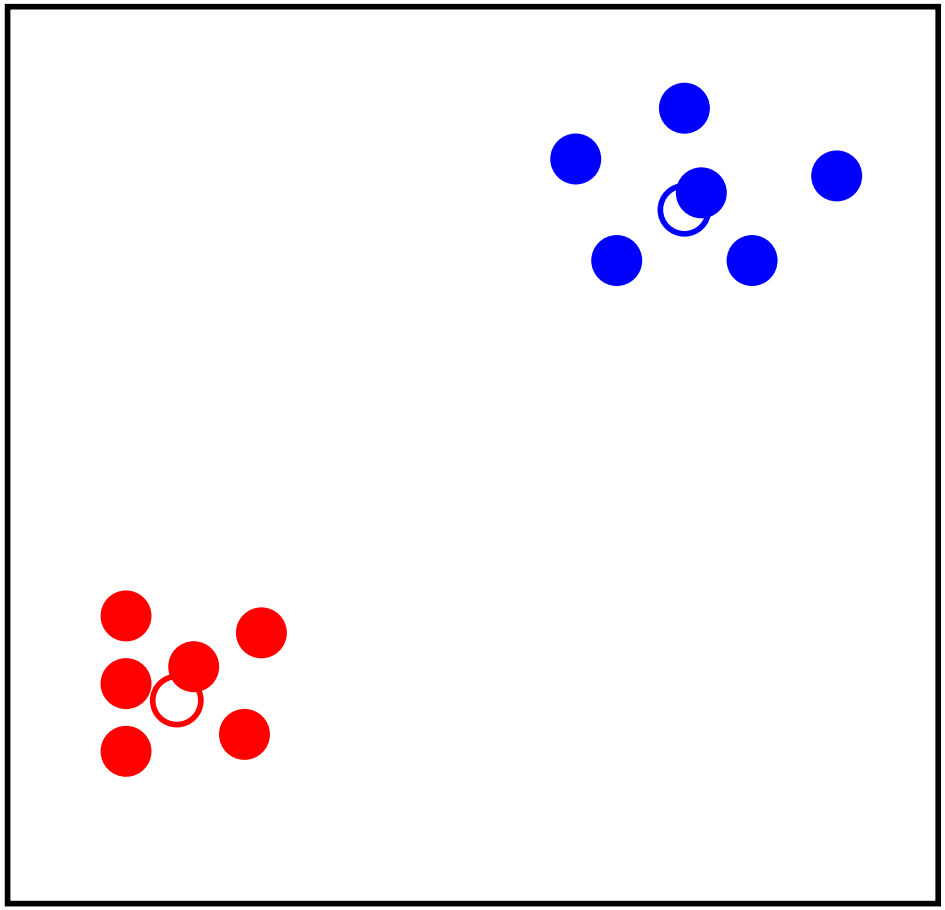
The k-means Algorithm

Re-assign points to groups



The k-means Algorithm

Re-compute group averages



The k-means Algorithm

```
Data: Data points Data[N]; Number of clusters K
Result: p, e(p), parameters that minimmise the least
        squares error, the error.
% Randomly assign each point to a cluster
for i ← 1 to N do
  | ClusterIndex[i] ← randint(1, K)
end
% Calculate the mean of each cluster
for i ← 1 to K do
  | ClusterMean[i] ← mean(Data[Cluster[i]])
end
% Iterate until clusters are stable
Converged ← False;
while Converged == False do
  | % Compute distance from every point to each
  |   cluster mean
  | for i ← 1 to K do
  |   | for j ← 1 to N do
  |   |   | dist[i][j] ← distance(ClusterMean[i], Data[j])
  |   | end
  | end
  | % Find the nearest mean for each data point
  | for i ← 1 to N do
  |   | for j ← 1 to K do
  |   |   | NewClusterIndex[i] =
  |   |   | nearest(Data[i], ClusterMean[j])
  |   | end
  | end
  | % Generate new clusters
  | for j ← 1 to K do
  |   | ClusterMean[i] ←
  |   | mean(Data[NewClusterIndex == i])
  | end
  | % Check to see if cluster membership has
  |   changed
  | Converged ← True
  | if NewClusterIndex != ClusterIndex then
  |   | Converged ← False
  | end
  | % Replace old clusters with new ones
  | ClusterIndex ← NewClusterIndex
  | ClusterMean ← NewClusterMean
end
```

Algorithm 1: The k-Means Algorithm.

Agglomerative Clustering

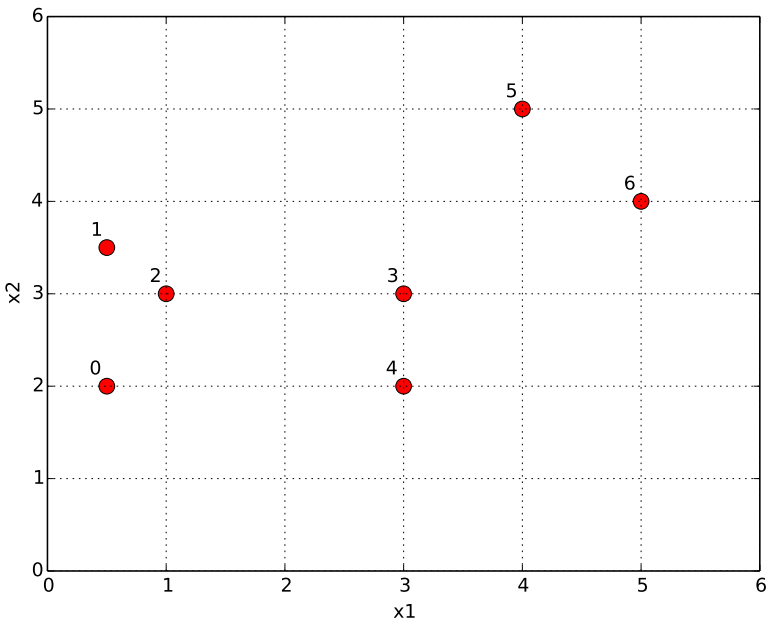
- k-means is a top-down algorithm.
- Clusters are "flat" with no structure.
- Change $k \rightarrow$ must recluster.
- Strict convex partitioning not appropriate for many types of data.
- Hierarchical clustering is an alternative "bottom-up" strategy
- Sequentially pair closest point together.

Agglomerative Clustering

1) Compute distances between all pair of data points
2) Find the closest pair
3) Remove this pair from the dataset and replace it with the grouped pair.
4) Recalculate distances between all points and the new group using a linkage strategy.
5) Go to 2) and continue grouping until all points are grouped

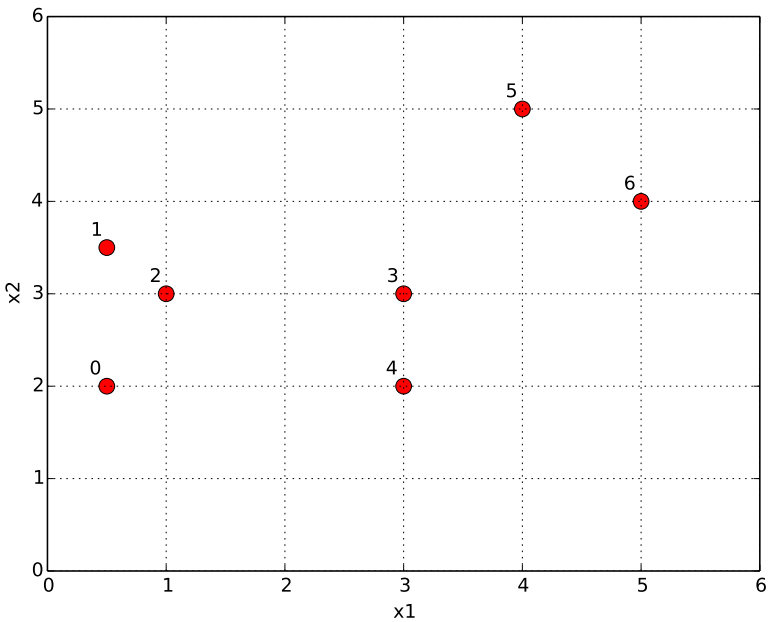
Introduction
Measures of Similarity
Normalisation
Linkage
The k-means Algorithm
Agglomerative Clustering
Other Clustering Methods
References

Agglomerative Clustering



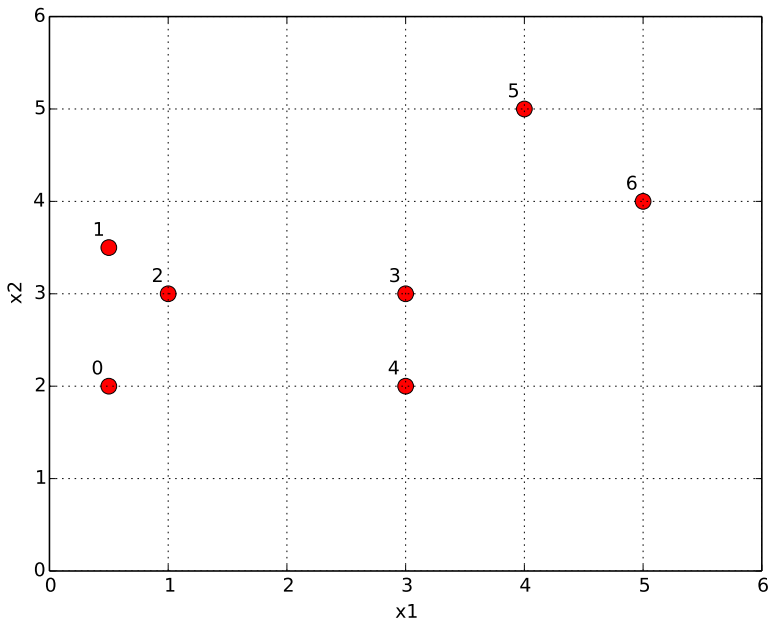
List of points 0,1,2,3,4,5,6

Agglomerative Clustering



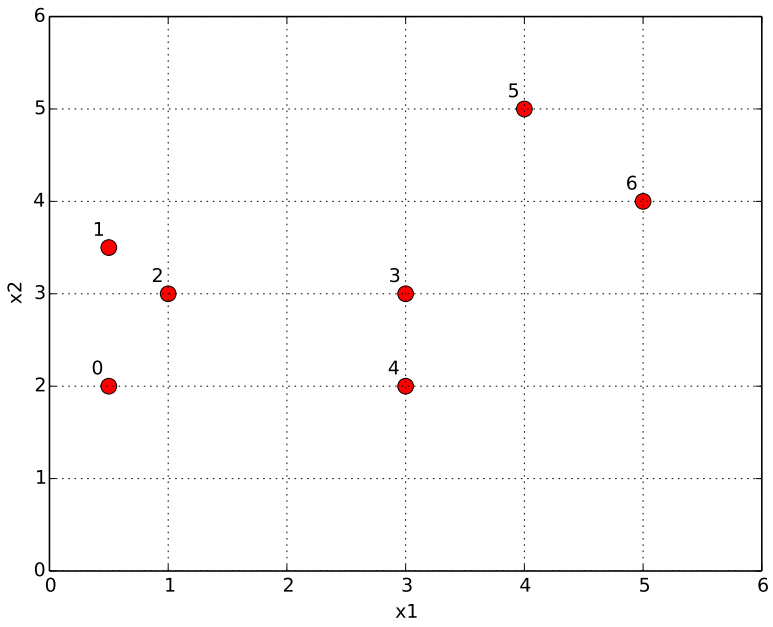
List of points		0,1,2,3,4,5,6
Group 1 with 2	$\mapsto$	0,3,4,5,6,(1,2)

Agglomerative Clustering



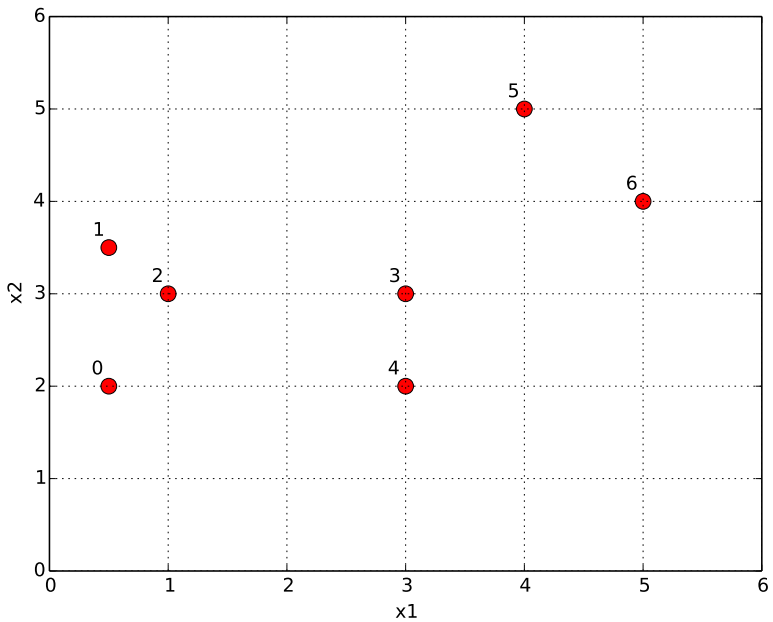
List of points		0,1,2,3,4,5,6
Group 1 with 2	$\mapsto$	0,3,4,5,6,(1,2)
Group 3 with 4	$\mapsto$	0,5,6,(1,2),(3,4)

Agglomerative Clustering



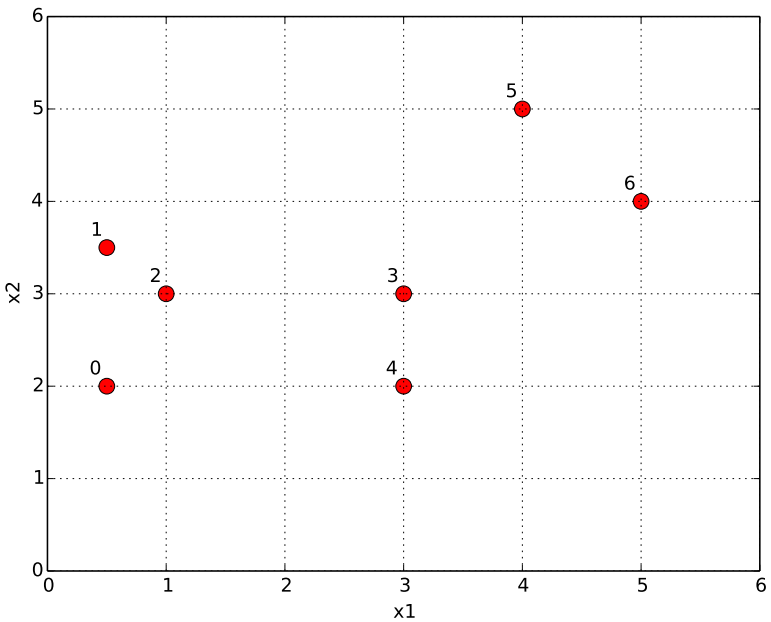
List of points		0,1,2,3,4,5,6
Group 1 with 2	$\mapsto$	0,3,4,5,6,(1,2)
Group 3 with 4	$\mapsto$	0,5,6,(1,2),(3,4)
Group 0 with (1,2)	$\mapsto$	5,6,(3,4),(0,(1,2))

Agglomerative Clustering



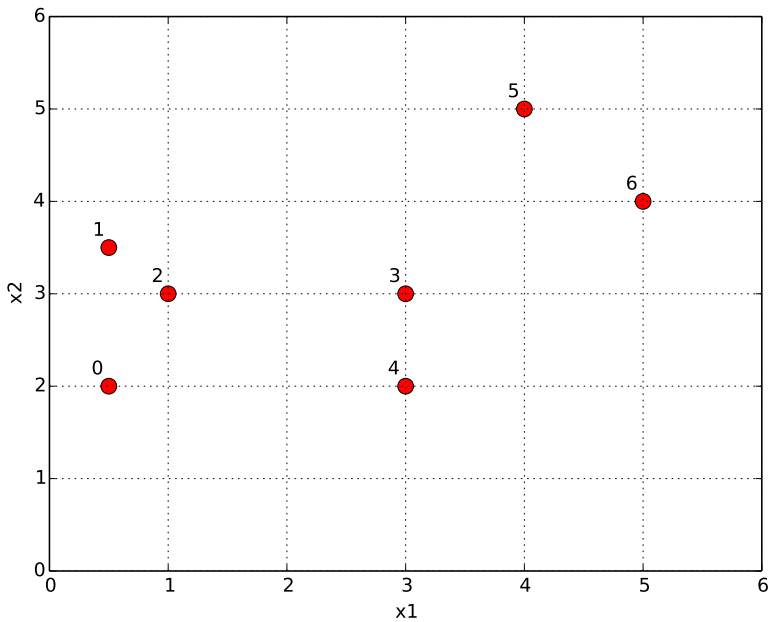
List of points		0,1,2,3,4,5,6
Group 1 with 2	$\mapsto$	0,3,4,5,6,(1,2)
Group 3 with 4	$\mapsto$	0,5,6,(1,2),(3,4)
Group 0 with (1,2)	$\mapsto$	5,6,(3,4),(0,(1,2))
Group 5 with 6	$\mapsto$	(3,4),(0,(1,2)),(5,6)

Agglomerative Clustering



List of points		0,1,2,3,4,5,6
Group 1 with 2	$\mapsto$	0,3,4,5,6,(1,2)
Group 3 with 4	$\mapsto$	0,5,6,(1,2),(3,4)
Group 0 with (1,2)	$\mapsto$	5,6,(3,4),(0,(1,2))
Group 5 with 6	$\mapsto$	(3,4),(0,(1,2)),(5,6)
Group (3,4) with (0,(1,2))	$\mapsto$	(5,6),((3,4),(0,(1,2)))

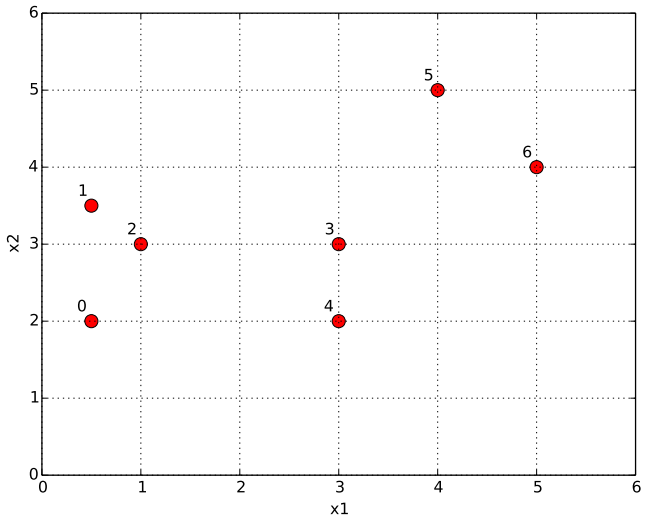
Agglomerative Clustering

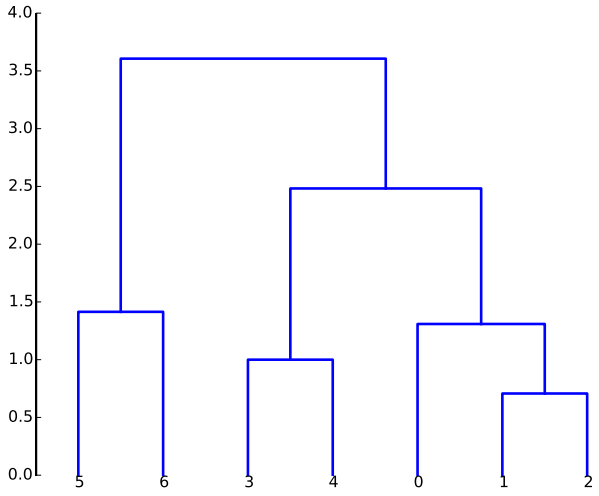


List of points		0,1,2,3,4,5,6
Group 1 with 2	$\mapsto$	0,3,4,5,6,(1,2)
Group 3 with 4	$\mapsto$	0,5,6,(1,2),(3,4)
Group 0 with (1,2)	$\mapsto$	5,6,(3,4),(0,(1,2))
Group 5 with 6	$\mapsto$	(3,4),(0,(1,2)),(5,6)
Group (3,4) with (0,(1,2))	$\mapsto$	(5,6),((3,4),(0,(1,2)))
Group (5,6) with ((3,4),(0,(1,2)))	$\mapsto$	((5,6),((3,4),(0,(1,2))))

Agglomerative Clustering

- Represent as a **dendrogram** where the height of each "junction" in the dendrogram represents the distance
  - Eg. $(3, 4)$ join at height 1.





Agglomerative Clustering

- Generates all numbers of clusters.
  - Cut horizontally across the dendrogram.
- But many computations
  - k-means: N points, K cluster $\rightarrow N \times K$ distances
  - Hierarchical: "choose 2 from N" $= \frac{n!}{2\,(n-2)!}$ calculations just for first pairing.
- k-means faster if you know how many clusters, but if you don't will have to run multiple times and for multiple repeats.
- They will NOT normally give identical results: choice depends on nature of data.

Other Clustering Methods

- Two of many available clustering techniques, but common choices.
- Still many free choices:
  - How many clusters? Which distance metric? What linkage method?
- Optimal combination often trial-and-error, but some "internal" validation measures can be used.
- Should understand the assumptiona and types of cluster you can get before choosing.
- Many clustering tools are available.
  - Matlab Statistics Toolbox includes many
  - ELKI (Environment for Developing KDD-Applications Supported by Index-Structures) from `http://elki.dbs.ifi.lmu.de/` includes many advanced techniques e.g. DBSCAN[EKSX96].

[EKSX96] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Kdd*, volume 96, pages 226–231, 1996.